

【项目】网约车复购率优化策略 - 钱鸿林

复购率指用户完成一笔订单后，其中再次下单的用户占用户总数的比例，通常分析7天/14天/30天内再次下单的用户占比。

回顾运营数据时发现完成首单的用户7天复购率在近期呈现下降趋势，分析发现在APP端完成首单的用户占比减少，在支付宝、微信平台完成首单的用户占比增长明显，而支付宝、微信平台完成首单的用户7天复购率水平较低，拉低了整体的首单用户7天复购率。

为提高支付宝、微信平台完成首单的用户7天复购率，平台决定对在支付宝、微信平台完成首单、且未复购的用户，在首单完成后第二天，发放短信“【**出行】送您最高10元立减券，X月X日前有效，点击<http://...>了解优惠详情。”，发放短信的时间与首单的时间相同，X月X日为首单日期后的第7天日期，用户点击短信链接，链接页面信息是提醒用户进入平台查看优惠券；用户在优惠券有效期内进入平台，平台弹窗提醒获得优惠券（显示具体的优惠券金额，优惠券金额随机，范围是4-10元）。

平台随机选择了一批在支付宝、微信平台完成首单、且未复购的用户：其中一部分发送优惠券到账短信，发送短信时优惠券自动发送到账户中；剩余部分不发送短信，且不发优惠券。

数据分析师需分析该优惠券策略的效果

数据字段：

- user_id：用户ID
- is_send_message：是否发优惠券短信，1为发短信、0为不发短信
- coupon_amount：发放的优惠券金额，0为不发优惠券
- order_waiting_time：首笔订单发起订单到上车的等待时间
- order_trip_distance：首笔订单行程距离
- order_rate：首笔订单评分
- order_platform：首笔订单打车平台（微信、支付宝）
- register_day：注册天数
- first_order_coupon：首单使用优惠券
- is_click_url：是否点击短信链接

- is_enter_platform: 7天内是否进入平台
- Is_repurchase: 7天内是否复购



数据分析师需分析：

1) 发放优惠券是否提高了在支付宝、微信平台完成首单用户的7天复购率?

样本筛选：只保留「在支付宝/微信完成首单」且有7天复购标记的用户。

分组：

- 实验组 (Treatment)：收到短信，且优惠券已自动到账
- 对照组 (Control)：未收到短信，且未发券

计算比例：分别计算两组 7 天复购率（各组人群特征一致即可）

- p^1 =实验组7天复购人数实验组样本数
- p^0 =对照组7天复购人数对照组样本数

输出提升值。

统计检验

- 用 **比例差异的显著性检验 (Z-test)** 验证差异是否显著

H_0 : 发券组复购率 = 不发券组复购率

H₁: 发券组复购率 不等于 不发券组复购率

Code block

```
1 import pandas as pd
2 import numpy as np
3 from statsmodels.stats.proportion import proportions_ztest
4
5 # 样本筛选：用户在 支付宝/微信平台完成首单 且 有 7 天复购标记
6 df_filtered = df[df['order_platform'].isin(['支付宝', '微信']) &
7                 (df['is_repurchase'].notnull())].copy()
8
9 treatment = df_filtered[(df_filtered['is_send_message'] == 1) &
10                        (df_filtered['coupon_amount'] > 0)]
```

```

9  control = df_filtered[(df_filtered['is_send_message'] == 0) &
    (df_filtered['coupon_amount'] == 0)]
10
11  p1 = treatment['is_repurchase'].mean()
12  p0 = control['is_repurchase'].mean()
13  uplift = p1 - p0
14
15  print(f"实验组复购率 (p1): {p1*100:.2f}%")
16  print(f"对照组复购率 (p0): {p0*100:.2f}%")
17  print(f"提升值 (uplift): {uplift*100:.2f}%")
18
19  # Z-test 验证显著性
20  count = np.array([treatment['is_repurchase'].sum(),
    control['is_repurchase'].sum()])
21  nobs = np.array([treatment.shape[0], control.shape[0]])
22  stat, pval = proportions_ztest(count, nobs) # 默认双侧检验
23
24  print(f"Z 统计量: {stat:.2f}")
25  print(f"P 值: {pval:.4e}")
26
27  # 判断显著性
28  if pval < 0.05:
29      print("结论: 差异显著, 发放优惠券显著提升了7天复购率。")
30  else:
31      print("结论: 差异不显著, 发放优惠券没有显著提升7天复购率。")

```

实验组复购率 (p1): 10.38%

对照组复购率 (p0): 6.47%

提升值 (uplift): 3.91%

Z 统计量: 38.60

P 值: 0.0000e+00

结论: 差异显著, 发放优惠券显著提升了7天复购率。

2) 识别出哪类用户发优惠券短信无法提高复购率, 用于节省短信费用?

用 uplift tree classifier 按“复购概率提升”来筛人

样本集合: 在支付宝/微信完成首单、且首单+1天仍未复购并被随机分到实验/对照的用户。

处理标记 ☐ t: 实验组=1 (发短信+自动下发券), 对照组=0 (什么都不发)。

结果标签 ☐ y: 是否在首单后第2~7天 (含) 发生复购 (1/0)。

特征 X：首单时可得的`用户与行为特征`（注册到首单时长、首单金额/品类、下单时间段、城市等级、渠道、设备等）。**不要用**“点击短信/见到弹窗/优惠券金额”等仅存在于实验组的变量。

训练完后拿到每个用户的 **预测 uplift**: $\Delta p = p(\text{复购}|\text{发短信}) - p(\text{复购}|\text{不发短信})$ 。筛选规则 (省短信费) :

- $\Delta p \leq 0$ 列入“不发短信”名单（这类人发了也不增反降）。

Code block

```

1  import numpy as np
2  import pandas as pd
3  from sklearn.model_selection import train_test_split
4  from sklearn.preprocessing import OneHotEncoder, StandardScaler
5  from sklearn.compose import ColumnTransformer
6
7  from causalm1.inference.tree import UpliftTreeClassifier
8  from causalm1.metrics.visualize import plot_qini
9  from causalm1.metrics import qini_score
10
11 import matplotlib.pyplot as plt
12
13 # 1) 样本集合
14 df = df[df['order_platform'].isin(['支付宝', '微信'])].copy()
15
16 df['y'] = df['is_repurchase'].astype(int)
17 df['treatment'] = np.where(df['is_send_message'] == 1, 'send_message',
18                             'no_message')
19
20 # 2) 特征工程
21 num_features = ['register_day', 'order_waiting_time',
22                 'order_trip_distance', 'order_rate']
23 cat_features = ['order_platform', 'first_order_coupon']
24
25 X = df[num_features + cat_features].copy()
26 y = df['y'].values
27 treatment = df['treatment'].values
28 user_ids = df['user_id'].values
29
30 preprocessor = ColumnTransformer(
31     transformers=[
32         ('num', StandardScaler(), num_features),
33         ('cat', OneHotEncoder(handle_unknown='ignore',
34                                sparse_output=False), cat_features)
35     ]
36 )

```

```

32     ]
33 )
34
35 X_mat = preprocessor.fit_transform(X)
36
37 # 3) 划分训练/测试集
38 X_train, X_test, y_train, y_test, t_train, t_test, uid_train, uid_test =
train_test_split(
39     X_mat,
40     y,
41     treatment,
42     user_ids,
43     test_size=0.2,
44     random_state=42,
45     stratify=treatment
46 )
47
48 # 4) 训练 uplift tree
49 uplift_model = UpliftTreeClassifier(
50     control_name='no_message',
51     max_depth=5,
52     min_samples_leaf=50,
53 )
54
55 uplift_model.fit(X=X_train, treatment=t_train, y=y_train)
56
57 # 5) uplift 预测
58 proba = uplift_model.predict(X_test)
59
60 # proba[:, 0] = P(repurchase=1 | no_message)
61 # proba[:, 1] = P(repurchase=1 | send_message)
62
63 # uplift = P(repurchase = 1 | send_message) - P(repurchase = 1 |
no_message)
64 uplift_vals = proba[:, 1] - proba[:, 0]
65
66 result_df = pd.DataFrame({
67     'P_no_message': proba[:, 0],      # 用户在“不发短信”组下被预测为复购
        (order=1) 的概率 (对照组预测概率)
68     'P_send_message': proba[:, 1],    # 用户在“发短信”组下被预测为复购
        (order=1) 的概率 (试验组预测概率)
69     'model_uplift': uplift_vals,      # 模型预测的 uplift 值, 表示因干预带
        来的增量概率
70     'treatment': t_test,              # 测试集中每个样本的实际分组标签
71     'actual_outcome': y_test          # 测试集中每个样本的真实结果
        (is_repurchase, 0/1 表示是否复购)
72 })

```

```

73
74 # 映射 treatment 为 0/1
75 result_df['treatment_map'] = df['treatment'].map({'no_message': 0,
76 'send_message': 1}).astype(int)
77
78 print(f"满足 model_uplift <= 0 的用户数: {(result_df['model_uplift'] <=
79 0).sum()} / {len(result_df)}")

```

满足 model_uplift <= 0 的用户数: 52 / 159159

Code block

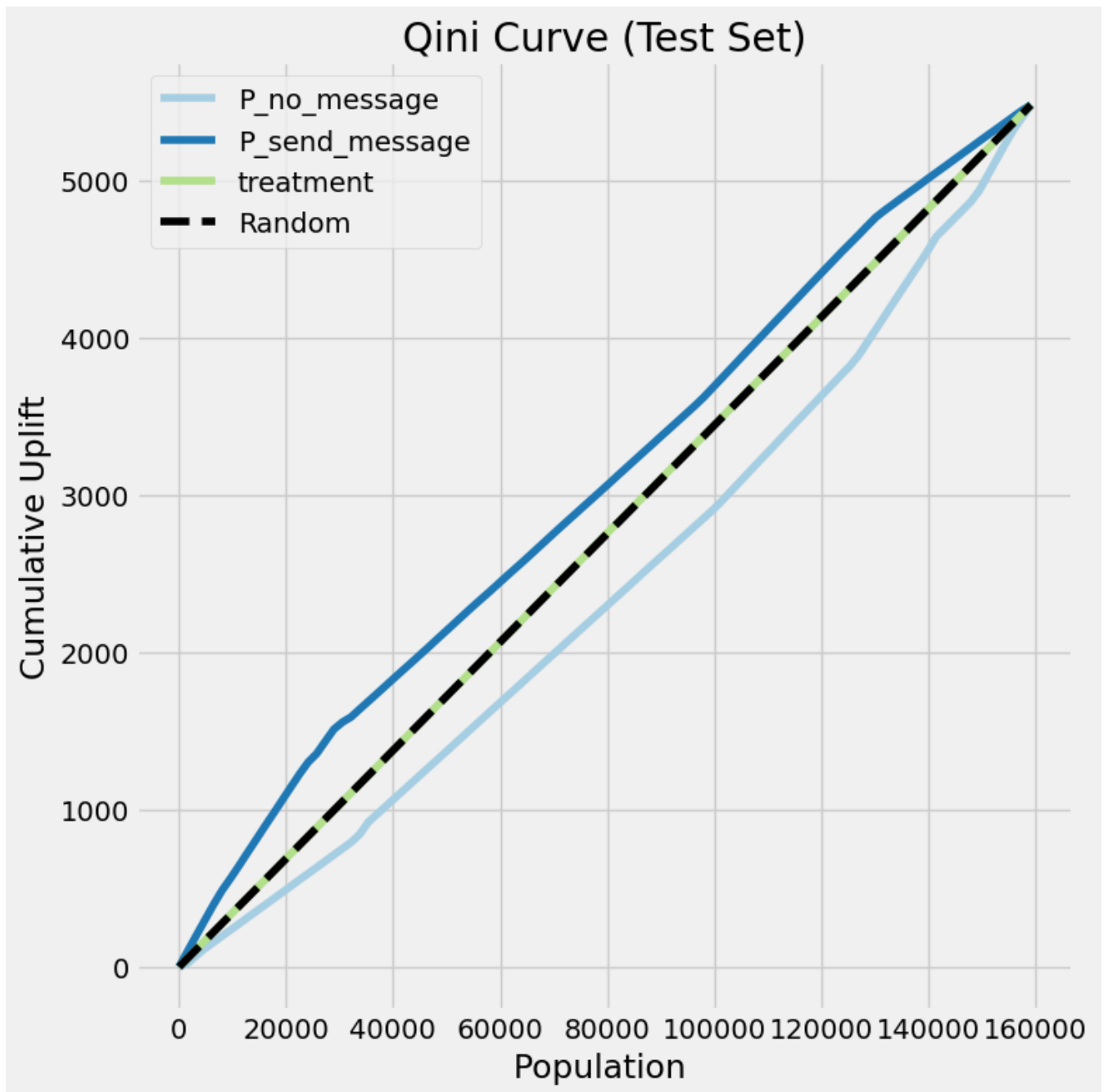
```

1  from causalmml.metrics.visualize import plot_qini
2  from causalmml.metrics import qini_score
3  import matplotlib.pyplot as plt
4
5  # 画 Qini 曲线
6  plt.figure(figsize=(13, 6), dpi=300)
7
8  plot_qini(
9      result_df,                                # DataFrame: 包含实际结果、处理分
      outcome_col='actual_outcome',              # str: 实际观测结果列名, 用于计算累
      treatment_col='treatment_map',             # str: 处理分组列名, 指示哪些样本接
      treatment_effect_col='model_uplift',        # str: 模型预测的个体提升效果列名
      normalize=False                           # bool: 是否对累积提升值进行归一
14 )
15 plt.title("Qini Curve (Test Set)")
16 plt.xlabel("Population ")
17 plt.ylabel("Cumulative Uplift")
18 plt.tight_layout()
19 plt.show()
20
21 # 计算 Qini score
22 score = qini_score(
23     result_df,
24     outcome_col='actual_outcome',
25     treatment_col='treatment_map',
26     treatment_effect_col='model_uplift',
27     normalize=False
28 )

```

29

```
30 print(f"模型的 Qini score: {score}")
```



模型的 Qini score:

P_no_message -342.484628

P_send_message 296.167839

treatment 0.537170

Code block

```
1  import numpy as np
2  import pandas as pd
3  from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
4  import warnings
5  warnings.filterwarnings("ignore")
6
7  # 分离数值 & OHE 特征
8  X_scaled      = preprocessor.transform(df)
9  n_num         = len(num_features)
10 X_num_scaled   = X_scaled[:, :n_num]
11 X_cat_ohe      = X_scaled[:, n_num:]
12
13 # 数值部分逆变换到“原始”尺度
14 # scaler       = preprocessor.named_transformers_['num']
15 X_num_orig     =
    preprocessor.named_transformers_['num'].inverse_transform(X_num_scaled)
16
17 # 横向拼接: 原始数值 + OHE 数值 (0/1)
18 X_rebuilt      = np.hstack([X_num_orig, X_cat_ohe])
19
20 # 构造特征名 (原数值名 + OHE 名)
21 cat_feature_names =
    preprocessor.named_transformers_['cat'].get_feature_names_out(cat_features)
22 feature_names   = num_features + list(cat_feature_names)
23
24 df_repr        = pd.DataFrame(X_rebuilt, columns=feature_names)
25
26 # 全数据计算 uplift、生成 mask
27 proba          = uplift_model.predict(X_scaled)                # 或
    .predict_proba
28 uplift_vals     = proba[:, 1] - proba[:, 0]
29 mask           = uplift_vals <= 0
30
31 result_df = pd.DataFrame({
32     'P_no_threshold': proba[:, 0],
33     'P_discount':     proba[:, 1],
34     'model_uplift':   uplift_vals
35 })
36
37 print(f'满足 model_uplift <= 0 的用户数: ' f'{{result_df["model_uplift"]
    <= 0).sum()}} / {{len(result_df)}}')
38
39
40 # 在“原始”特征上重训练决策树
```

```

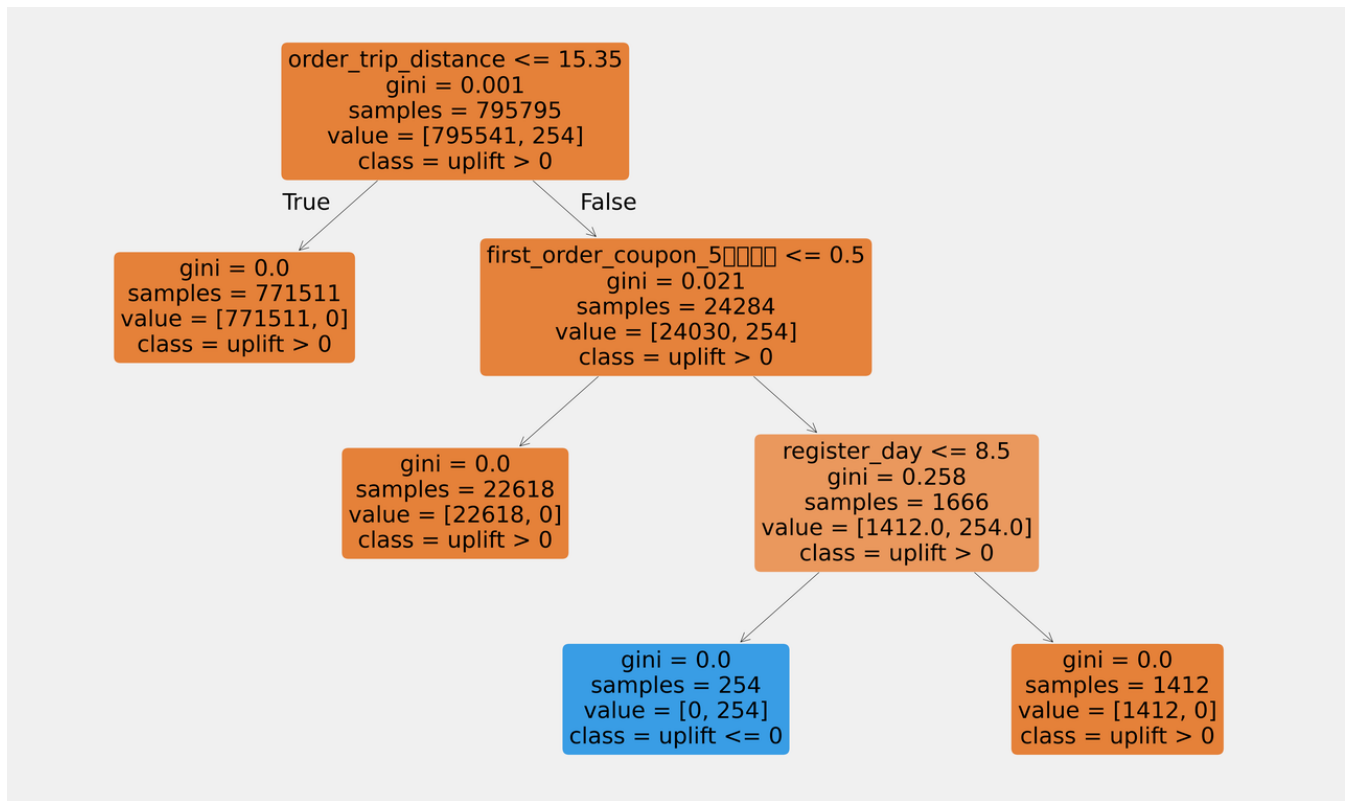
41 rule_tree      = DecisionTreeClassifier(max_depth=5, min_samples_leaf=50)
42 rule_tree.fit(df_repr, mask)
43
44 # mask = (uplift_vals <= 0) → 树学的是 哪些人发短信没用（不值得发）。
45
46 print(export_text(rule_tree, feature_names=feature_names))
47
48 plt.figure(figsize=(20, 12), dpi=300)
49 plot_tree(
50     rule_tree,
51     feature_names=feature_names,
52     class_names=['uplift > 0', 'uplift <= 0'], # 顺序严格对应 [0,1]
53     filled=True,
54     rounded=True
55 )
56
57 plt.tight_layout()
58 plt.show()

```

```

|--- order_trip_distance <= 15.35
| |--- class: False
|--- order_trip_distance > 15.35
| |--- first_order_coupon_5元立减券 <= 0.50
| | |--- class: False
| |--- first_order_coupon_5元立减券 > 0.50
| | |--- register_day <= 8.50
| | | |--- class: True
| | |--- register_day > 8.50
| | | |--- class: False

```



满足 $\text{model_uplift} \leq 0$ 的用户数：254 / 795795

总结成业务规则, 不值得发短信的人群:

下单距离 超过 15.35 公里, 并且 领取过 5 元首单优惠券, 并且 注册天数 在 8.5 天以内的用户。

3) 分析用户在哪个优惠券金额下的复购率最高?

1. 特征准备

- 选取用户画像和订单特征作为协变量 X 。对离散变量做 one-hot 编码（比如平台类型、首单是否有券）。
- T = 用户获得的优惠券金额（离散处理变量）。
- Y = 是否复购（二元结果）。

2. 定义因果森林模型

- `model_t`：预测给定用户画像时，平台会发多少金额的券。
- `model_y`：预测在 (T, X) 下，用户复购的概率。
- `CausalForestDML`：通过双重机器学习（DML）校正偏差，估计个体处理效应（ITE）。

3. 个体最优优惠券金额 (个性化推荐)

- 遍历候选优惠券金额（目前的金额 $t - 10$ 元）。
- 计算 **该用户在金额 t 相比 4 元时复购率提升多少**。
- 找到提升效果最大的金额 $\rightarrow$ 个体最优优惠券。

整体逻辑总结

- 用因果森林学习优惠券金额对复购率的因果效应。
- 得到个体最优优惠券金额 (ITE)，实现个性化优惠券分配。

Code block

```

1  from econml.dml import CausalForestDML
2  from sklearn.ensemble import RandomForestRegressor,
   RandomForestClassifier
3  from sklearn.model_selection import train_test_split
4  import matplotlib.pyplot as plt
5  from causalml.metrics.visualize import plot_qini
6  from causalml.metrics import qini_score
7  from tqdm import tqdm
8  import numpy as np
9
10 # 分析人群需要限制是接收了短信且7天内进入平台的用户，因为只有进入了平台才能看到优惠券金额 ! ! ! ! ! ! ! ! ! !
11 df_filtered = df[(df["is_send_message"] == 1) & (df["is_enter_platform"]
   == 1)]
12
13 X = df_filtered[[
14     "order_waiting_time",
15     "order_trip_distance",
16     "order_rate",
17     "order_platform",
18     "register_day",
19     "first_order_coupon"
20 ]]
21 X = pd.get_dummies(X, columns=["order_platform", "first_order_coupon"],
   drop_first=True)
22
23 T = df_filtered["coupon_amount"].values # 连续处理：优惠券金额
24 Y = df_filtered["is_repurchase"].values # 二分类：是否复购
25
26 X_train, X_test, T_train, T_test, Y_train, Y_test = train_test_split(X,
   T, Y, test_size=0.2, random_state=42)
27
28 est = CausalForestDML(
29

```

```

30     # model_t
31     # 拟合 T (优惠券金额) 和协变量 X 的关系。
32     # 相当于回答：“给定用户画像和订单特征，平台可能给他多少金额的券？”
33     # 用随机森林回归器来做，因为 T 是连续变量（金额）。
34
35     model_t=RandomForestClassifier( # 第一阶段：拟合 coupon_amount (处理变
    量 T) 的模型
36         n_estimators=50,
37         min_samples_leaf=10,
38         random_state=42
39     ),
40
41     # model_y
42     # 拟合 Y (是否复购) 和 (T, X) 的关系。
43     # 回归器预测的是 复购概率 (0-1 之间)，所以用回归而不是分类。
44
45     model_y=RandomForestClassifier( # 第二阶段：拟合 is_repurchase (结果变
    量 Y) 的模型
46         n_estimators=50,
47         min_samples_leaf=10,
48         random_state=42
49     ),
50
51     # CausalForestDML 本身的超参数
52     n_estimators=100,
53     min_samples_leaf=5,
54     discrete_treatment=True,
55     discrete_outcome=True,
56     random_state=42
57 )
58
59 est.fit(Y_train, T_train, X = X_train)
60

```

针对每个用户，寻找最优优惠券金额：

计算每个用户金额 vs 4 元的 复购uplift增量

```
eff = est.effect(X, T0=np.full(len(X), 4), T1=np.full(len(X), t))
```

特殊情况：如果所有 uplift <= 0，则选4元

```

1  # 针对每个用户，寻找最优优惠券金额
2  all_effects = {}
3
4  # 计算每个金额 vs 4 元的 uplift (批量)
5  for t in tqdm(range(5, 11)):
6      eff = est.effect(X, T0=np.full(len(X), 4), T1=np.full(len(X), t))
7      all_effects[t] = eff.ravel()
8
9  # est.effect = 因果森林输出，用于「预测  $\Delta$  uplift (效应)」。(复购率提升 或 下降了多少)
10 # 个体处理效应 (ITE) 模型估计的“每个用户拿到更高金额优惠券时复购率的提升效果”。
11 # 在 econml.CausalForestDML 里，est.effect(X_test) 默认比较的是：处理变量 T 增加一个单位 (+1) 对结果 Y 的影响。
12
13 # 也就是说：baseline = 用户自己在训练数据中的 实际金额
14 # T1 = baseline + 1
15 # 所以 te_pred[i] = 如果该用户的优惠券金额比他本来拿到的金额再多 1 元，复购概率会增加多少。
16
17 df_uplift = pd.DataFrame(all_effects, index=df_filtered["user_id"])
18 df_uplift.index.name = "user_id"
19
20 # 找到最优金额和最大 uplift
21 optimal_coupon = df_uplift.idxmax(axis=1) # 每行最大值的列名 (最佳金额)
22 predicted_uplift = df_uplift.max(axis=1) # 最大 uplift 值
23
24 # 如果所有 uplift <= 0, 则选4元
25 optimal_coupon = optimal_coupon.where(predicted_uplift > 0, 4)
26
27 # 最终结果表
28 df_optimal_vs4 = pd.concat([
29     df_filtered[["user_id"]].reset_index(drop=True),
30     pd.Series(T, name="actual_coupon"), # 实际发放金额
31     df_uplift.reset_index(drop=True), # 每个金额的 uplift, 一元一列
32     pd.Series(optimal_coupon.values, name="optimal_coupon_vs4"),
33     pd.Series(predicted_uplift.values, name="predicted_uplift_vs4")
34 ], axis=1)
35
36 df_optimal_vs4

```

部分结果如下：

	user_id	actual_coupon	5	6	7
0	795641589	8	-0.140928	-0.236254	-0.049155
1	809872094	9	0.096219	-0.027597	-0.054105
2	826052918	5	-0.082745	0.138214	0.085222
3	231904133	6	0.036501	0.260983	-0.026969
4	833002734	5	0.173408	0.048942	0.027179
...	...	...	...	...	...
156125	882937665	10	-0.037101	-0.072005	0.00668
156126	679123886	7	-0.160774	-0.028458	0.15824
156127	886955733	4	0.177878	0.168273	0.037205
156128	107749841	6	-0.148884	0.214376	0.190706
156129	141806071	4	0.063732	0.12204	0.240847

每个用户ID有原本分配的优惠券金额actual_coupon，同时通过CausalForestDML训练出了对比4元优惠券的最大复购增益的优惠券金额optimal_coupon_vs4，如果5元以上的策略都小于0，那就要选4元。

4) 相比于发最低的4元优惠券，其他金额的优惠券每多 1 元是否能得到一个成本效益比较高的优惠券金额推荐方案？

第一步：筛选分析人群

- 基于“问题 2”的 uplift 决策树，筛选出 值得发短信的人群（即短信能显著提升复购率的人）。
- 在上述人群中，再限定 接收了短信且 7 天内进入过平台的用户。理由：只有进入平台的用户才能看到并使用优惠券金额。
- 最终分析样本
 - $user_id \in \{ \text{应发短信人群} \mid n_is_send_message=1 \wedge n_is_enter_platform=1 \}$

第二步：针对每个用户的成本效益 ROI 分析

分析目标：在进入平台的“应发短信人群”中，对 每个用户 计算不同优惠券金额（5-10 元，相比 4 元基准）的 单位成本效益 ROI，从而得到个性化推荐的最优金额。

计算方法

1. 复购率提升 (uplift) 对于用户 i ，优惠券金额 t ($t \in \{5, \dots, 10\}$) 相比于 4 元的 uplift:

$$\Delta Y_{i,t} = \hat{Y}(X_i, T = t) - \hat{Y}(X_i, T = 4)$$

其中 $\hat{Y}(X_i, T)$ 表示模型预测的复购概率。

2. 单位成本 ROI

$$ROI_{i,t} = \frac{\Delta Y_{i,t}}{t - 4}$$

表示每多花 1 元券成本，为用户 i 带来的复购率提升。

3. 最优金额选择

- 对每个用户，找到使 $ROI_{i,t}$ 最大的金额 t ;
- 若所有 $\Delta Y_{i,t} \leq 0$ ，则说明该用户相比 4 元没有额外提升 → 推荐维持 4 元。

第三步：推荐方案

- 在“筛选的人群”中，比较 5-10 元的 ROI:
 - 若 5 元 ROI 最高 → 则推荐 5 元作为主推券额；
 - 若 6 元或 7 元 ROI 更优 → 则在预算允许时可作为替代方案；
 - 若 所有 $\Delta Y_{i,t} \leq 0$ → 表示该人群相比 4 元没有额外提升，应维持 4 元。

Code block

```
1  import numpy as np
2  import pandas as pd
3
4  # 筛选进入平台 & 值得发短信的人
5  df_target = df.copy()
6  df_target["uplift"] = uplift_vals    # 这是短信uplift模型的预测结果
7  df_target = df_target[
8      (df_target["is_send_message"] == 1) &
9      (df_target["is_enter_platform"] == 1) &
10     (df_target["uplift"] > 0)    # 值得发短信的人
11 ]
12
13 X_target = X.loc[df_target.index]    # 特征子集
14
15 # 计算每个用户在不同金额下 uplift
16 all_effects = {}
17 for t in range(5, 11):
18     eff = est.effect(
19         X_target,
20         T0=np.full(len(X_target), 4),    # baseline: 4元
21         T1=np.full(len(X_target), t)    # 处理: t元
22     )
23     all_effects[t] = eff.ravel()
24
25 df_uplift_user = pd.DataFrame(all_effects, index=df_target["user_id"])
26 df_uplift_user.index.name = "user_id"
27
28 # 计算每个用户的 ROI
29 df_roi_user = df_uplift_user.copy()
30 for t in range(5, 11):
31     df_roi_user[t] = df_roi_user[t] / (t - 4)    # 单位成本ROI
32
33 # 找到每个用户ROI最大值 和 对应的coupon value
34 best_coupon_by_roi = df_roi_user.idxmax(axis=1)
35 best_roi_value = df_roi_user.max(axis=1)
36
37 # 如果所有 uplift <= 0, 则回退到4元 (ROI=0)
38 mask_all_negative = (df_uplift_user.max(axis=1) <= 0)
39 best_coupon_by_roi[mask_all_negative] = 4
40 best_roi_value[mask_all_negative] = 0
41
42 df_optimal_user = pd.concat([
43     df_target[["user_id"]].reset_index(drop=True),
```

```

44     pd.Series(df_target["coupon_amount"].values, name="actual_coupon"),
45     df_roi_user.reset_index(drop=True),    # 每个金额的 ROI
46     pd.Series(best_coupon_by_roi.values, name="best_coupon_by_roi"),
47     pd.Series(best_roi_value.values, name="best_roi_value")
48 ], axis=1)
49
50 df_optimal_user

```

用户级别ROI推荐方案：

	user_id	actual_coupon	5	6
0	795641589	8	-0.140928	-0.118127
1	809872094	9	0.096219	-0.013799
2	826052918	5	-0.082745	0.069107
3	231904133	6	0.036501	0.130491
4	833002734	5	0.173408	0.024471
...	...	...	...	...
156080	882937665	10	-0.037101	-0.036002
156081	679123886	7	-0.160774	-0.014229
156082	886955733	4	0.177878	0.084136
156083	107749841	6	-0.148884	0.107188
156084	141806071	4	0.063732	0.06102

5) 设计AB实验，用于验证分析得到的需要发短信的目标用户，在发短信后复购率是否会提升，文字描述即可。

5.1 实验目的：

验证通过分析筛选出的“需要发短信的目标用户”，在接收到短信后，是否会在 7 天内提高复购率。

5.3 核心指标：

7天内复购率：首单后用户 7 天内再次下单的比率。

5.2 实验策略动作：

如何分组：对目标人群（四个细分 Segment1-4）使用随机数切分，1:1 分配至对照/实验组。

- 对符合分群条件的用户进行如下操作：

- **实验组**：首单次日同时间收到短信提醒 + 4-10 元优惠券（随机金额）
- **对照组**：不发短信、不发优惠券。

5.3 分流单位：

以用户（user_id）为分流单位，确保同一用户只进入实验组或对照组。

5.4 最小样本量：

- 7 天复购率的历史水平（如基线复购率 $p_0 \approx x\%$ ）
- MDE: 预期提升幅度（如 $\Delta = 2\% \sim 3\%$ ）
- 设定显著性水平 $\alpha = 0.05$
- 检验效力 $1 - \beta = 0.8$
- 计算得到每组至少需要 N 人。

5.5 实验假设与检验方法：

p_1 : 定义为实验组（发短信和优惠券）用户的复购率

p_2 : 定义为对照组（不发短信和优惠券）用户的复购率

- 原假设 (H_0): $p_1 - p_2 = 0$
- 备择假设 (H_1): $p_1 - p_2 \neq 0$ （双侧检验）

设显著性水平 $\alpha = 0.05$ ，检验功效 $1 - \beta = 0.8$

检验方法：

- 双侧两样本比例 z 检验：

$$z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}(1 - \hat{p})\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}, \quad \hat{p} = \frac{k_1 + k_2}{n_1 + n_2}$$

- 拒绝域： $|z| > z_{1-\alpha/2} = 1.96$ ($\alpha = 0.05$)
- 若 $|z| > 1.96$ ，则拒绝 H_0 ，认为 p_1 与 p_2 存在显著差异。

一、背景和结论

- 问题发现

a. 最近数据回顾显示，整体首单用户的7天复购率出现下降。进一步拆分渠道发现：

- APP端首单用户占比下降，支付宝／微信端首单用户占比上升
- 支付宝／微信端首单用户的7天复购率明显低于APP端

说明：渠道结构变化导致整体7天复购率被拉低。

- 改进目标

- 提高“支付宝／微信端首单用户”的7天复购率，从而拉动整体首单7天复购率。

- 发券方案设计

a. 目标人群：

- 在支付宝或微信平台完成首单 且 截至首单后第2天尚未发生复购

b. 干预内容：

- 在“首单完成时刻”的次日同一时段，向目标人群发送短信：

【**出行】送您最高10元立减券，X月X日前有效，点击 <http://...> 了解优惠详情。

- 其中 X月X日 = “首单日 + 7天”，即优惠券有效期截至第7天24:00。

c. 优惠券发放：

- 平台随机选择了一批在支付宝、微信平台完成首单、且未复购的用户，
 - 其中一部分发送优惠券到账短信，发送短信时优惠券自动发送到账户中
 - 剩余部分不发送短信，且不发优惠券。

- 结论与建议

发放优惠券确实提高了在支付宝、微信平台完成首单用户的7天复购率；

但对于下单距离 超过 15.35 公里，并且 领取过 5 元首单优惠券，并且 注册天数 在 8.5 天以内 的用户发优惠券短信无法提高复购率，需要节省短信费用。

最后通过因果森林，识别出每个用户在特定优惠券金额下的最高复购率，同时考虑发放优惠券的性价比，分析每个用户优惠券金额每增加一块钱能带来多少复购率，哪个金额下的单位复购率提升最高。

最终识别出来用户个性化的优惠券推荐方案。

二、分析过程和数据表现

1. 核心分析思路

通过因果推断，得到每个用户个性化的优惠券推荐方案，以此提升首单用户的7天复购率；最后通过 ab test验证结果的显著性

2. 每一步分析动作

a. 发放优惠券是否提高了在支付宝、微信平台完成首单用户的7天复购率

- 发短信和优惠券的实验组复购率：10.38%
- 不发短信且不发优惠券的对照组复购率：6.47%
- 提升值 (uplift): 3.91%，Z 统计量: 38.60，P 值: 0.0000e+00
- 结论: 差异显著，发放优惠券显著提升了7天复购率。

2. 识别出哪类用户发优惠券短信无法提高复购率，用于节省短信费用

用 uplift tree classifier 按“复购概率提升”来筛人

样本集合：在支付宝/微信完成首单、且首单+1天仍未复购并被随机分到实验/对照的用户。

处理标记 t ：实验组=1（发短信+自动下发券），对照组=0（什么都不发）。

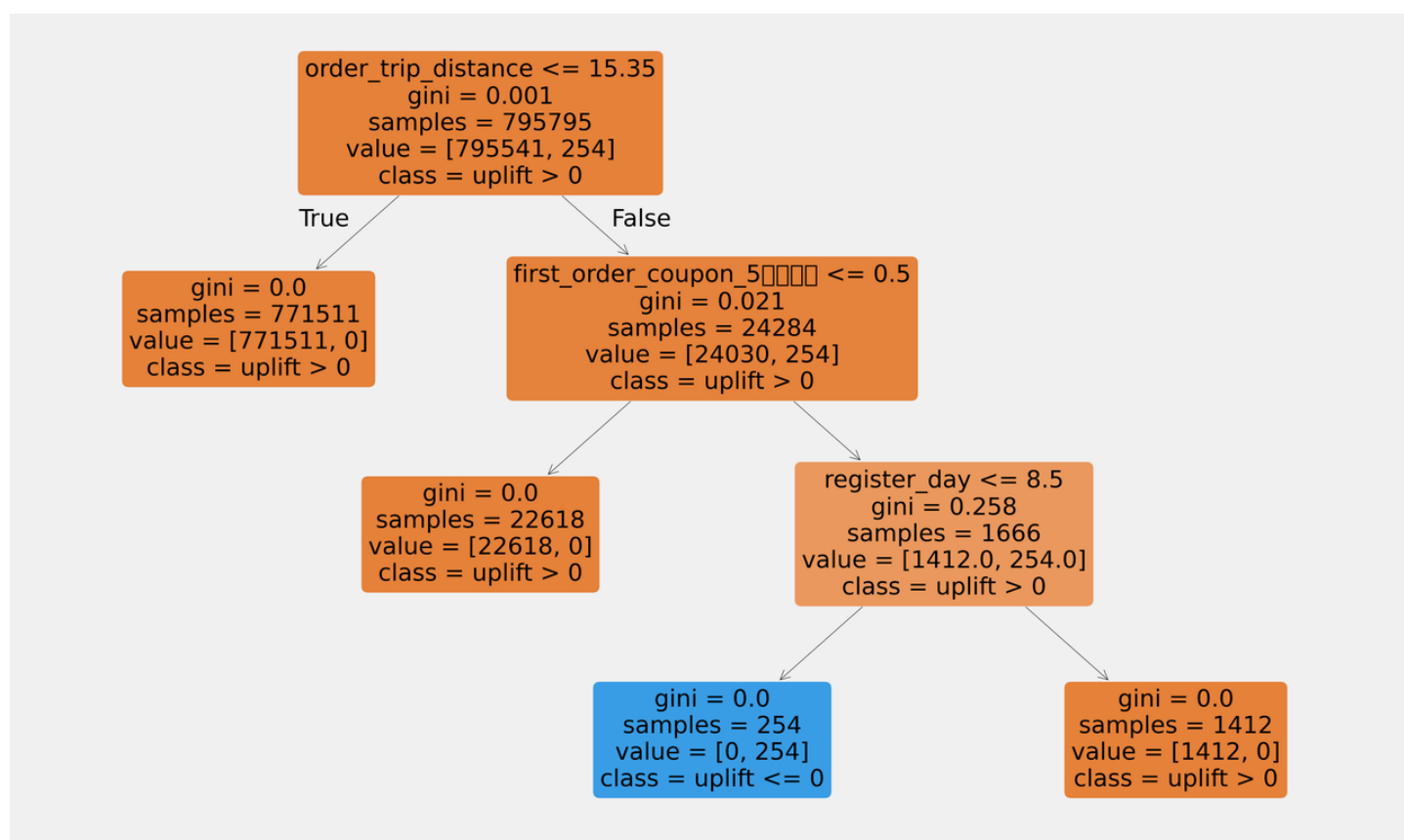
结果标签 y ：是否在首单后第2~7天（含）发生复购（1/0）。

特征 X ：首单时可得的用户与行为特征（注册到首单时长、首单金额/品类、下单时间段、城市等级、渠道、设备等）。**不要用**“点击短信/见到弹窗/优惠券金额”等仅存在于实验组的变量。

训练完后拿到每个用户的 **预测 uplift**: $\Delta p = p(\text{复购}|\text{发短信}) - p(\text{复购}|\text{不发短信})$ 。筛选规则（省短信费）：

- $\Delta p \leq 0$ 列入“不发短信”名单（这类人发了也不增反降）。

```
|--- order_trip_distance <= 15.35
| |--- class: False
|--- order_trip_distance > 15.35
| |--- first_order_coupon_5元立减券 <= 0.50
| | |--- class: False
| | |--- first_order_coupon_5元立减券 > 0.50
| | | |--- register_day <= 8.50
| | | | |--- class: True
| | | |--- register_day > 8.50
| | | | |--- class: False
```



总结成业务规则, 不值得发短信的人群:

下单距离 超过 15.35 公里, 并且 领取过 5 元首单优惠券, 并且 注册天数 在 8.5 天以内 的用户。

c. 分析用户在哪个优惠券金额下的复购率最高

1. 特征准备

- 选取用户画像和订单特征作为协变量 X 。对离散变量做 one-hot 编码（比如平台类型、首单是否有券）。
- T = 用户获得的优惠券金额（离散处理变量）。
- Y = 是否复购（二元结果）。

2. 定义因果森林模型

- `model_t`：预测给定用户画像时，平台会发多少金额的券。
- `model_y`：预测在 (T, X) 下，用户复购的概率。
- `CausalForestDML`：通过双重机器学习（DML）校正偏差，估计个体处理效应 (ITE)。

3. 个体最优优惠券金额 (个性化推荐)

- 遍历候选优惠券金额（目前的金额 $t - 10$ 元）。
- 计算 **该用户在金额 t 相比 4 元时复购率提升多少**。
- 找到提升效果最大的金额 $\rightarrow$ 个体最优优惠券。

整体逻辑总结

- 用因果森林学习优惠券金额对复购率的因果效应。
- 得到个体最优优惠券金额 (ITE)，实现个性化优惠券分配。

部分结果如下：

	user_id	actual_coupon	5	6	7
0	795641589	8	-0.140928	-0.236254	-0.049155
1	809872094	9	0.096219	-0.027597	-0.054105
2	826052918	5	-0.082745	0.138214	0.085222
3	231904133	6	0.036501	0.260983	-0.026969
4	833002734	5	0.173408	0.048942	0.027179
...	...	...	...	...	...
156125	882937665	10	-0.037101	-0.072005	0.00668
156126	679123886	7	-0.160774	-0.028458	0.15824
156127	886955733	4	0.177878	0.168273	0.037205
156128	107749841	6	-0.148884	0.214376	0.190706
156129	141806071	4	0.063732	0.12204	0.240847

每个用户ID有原本分配的优惠券金额actual_coupon，同时通过CausalForestDML训练出了对比4元优惠券的最大复购增益的优惠券金额optimal_coupon_vs4，如果5元以上的策略都小于0，那就要选4元。

d. 相比于发最低的4元优惠券，其他金额的优惠券每多 1 元是否能得到一个成本效益比较高的优惠券金额推荐方案

第一步：筛选分析人群

- 基于“问题 2”的 uplift 决策树，筛选出 值得发短信的人群（即短信能显著提升复购率的人）。
- 在上述人群中，再限定 接收了短信且 7 天内进入过平台的用户。理由：只有进入平台的用户才能看到并使用优惠券金额。
- 最终分析样本
 - $user_id \in \{ \text{应发短信人群} \cap is_send_message=1 \cap is_enter_platform=1 \}$

第二步：针对每个用户的成本效益 ROI 分析

分析目标：在进入平台的“应发短信人群”中，对 每个用户 计算不同优惠券金额（5-10 元，相比 4 元基准）的 单位成本效益 ROI，从而得到个性化推荐的最优金额。

计算方法

1. 复购率提升 (uplift) 对于用户 i ，优惠券金额 t ($t \in \{5, \dots, 10\}$) 相比于 4 元的 uplift:

$$\Delta Y_{i,t} = \hat{Y}(X_i, T = t) - \hat{Y}(X_i, T = 4)$$

其中 $\hat{Y}(X_i, T)$ 表示模型预测的复购概率。

2. 单位成本 ROI

$$ROI_{i,t} = \frac{\Delta Y_{i,t}}{t - 4}$$

表示每多花 1 元券成本，为用户 i 带来的复购率提升。

3. 最优金额选择

- 对每个用户，找到使 $ROI_{i,t}$ 最大的金额 t ;
- 若所有 $\Delta Y_{i,t} \leq 0$ ，则说明该用户相比 4 元没有额外提升 → 推荐维持 4 元。

第三步：推荐方案

- 在“筛选的人群”中，比较 5-10 元的 ROI:
 - 若 5 元 ROI 最高 → 则推荐 5 元作为主推券额；
 - 若 6 元或 7 元 ROI 更优 → 则在预算允许时可作为替代方案；
 - 若 所有 $\Delta Y_{i,t} \leq 0$ → 表示该人群相比 4 元没有额外提升，应维持 4 元。

用户级别ROI推荐方案：

	user_id		actual_coupon		5
	0	795641589	8	-0.140928	-0.118
	1	809872094	9	0.096219	-0.013
	2	826052918	5	-0.082745	0.069
	3	231904133	6	0.036501	0.130
	4	833002734	5	0.173408	0.024
	...	...	...	...	
	156080	882937665	10	-0.037101	-0.036
	156081	679123886	7	-0.160774	-0.014
	156082	886955733	4	0.177878	0.084
	156083	107749841	6	-0.148884	0.107
	156084	141806071	4	0.063732	0.06

三、复盘空间

1、之前做的部分有哪些是可以优化的

a. 数据与特征

- 引入更多用户行为特征：历史消费频次/金额、活跃天数等。

b. 模型与评估

- 比较多种 Uplift 算法：Uplift XGBoost 等
- 参数优化：网格搜索，optuna 参数优化

c. 个体推荐解释性

- 优化：结合 feature importance，输出**关键特征**

2、未来还可以做些什么，来进一步解决业务问题

基于业务问题是提高复购率和控制优惠券营销成本，实施优惠券策略优化

o 人群分层策略

- 对 ROI 推荐结果进一步分层：
 - 高 ROI + 高 uplift → 优先投放
 - 低 ROI + 负 uplift → 不投放
 - 中间群体 → 可以测试其他运营手段（积分、话术、定向推送）。

